

TECHNICAL WHITEPAPER — v1.0 — JUNE 2026

VEIL PROTOCOL

A quantum-resistant, self-sovereign AI neobank
built on FIPS 203 (ML-KEM-768) and FIPS 204 (ML-DSA-65)



TABLE OF CONTENTS

Abstract	3
1. Introduction	3
1.1 Harvest-Now-Decrypt-Later Threat	3
1.2 NIST PQC Standards (2024)	4
1.3 Why Crypto Wallets Are Vulnerable	4
2. Architecture Overview	5
3. Biometric SSI	5
3.1 Fuzzy Extractor	5
3.2 HKDF-SHA3-512 Key Derivation	6
3.3 Social Recovery (SSS)	6
4. PQCTransport	6
4.1 ML-KEM-768 (FIPS 203)	7
4.2 ML-DSA-65 (FIPS 204)	7
4.3 Hybrid KEM-DEM Construction	7
4.4 Replay Window Protection	8
5. Ghost AI Agent	8
5.1 DistilBERT Intent Classifier	8
5.2 De-identification Pipeline	9
5.3 Tiered Pricing	9
5.4 FHE Circle Execution (Octra)	9
6. x402-pqc Payment Standard	10
6.1 Protocol Flow	10
6.2 AI Agent Discovery via A2A	10
6.3 x402 Foundation Proposal #2664	11
7. x402PQCPayments Contract	11
8. Token Economics	12
8.1 Allocation Table	12
8.2 Revenue-Backed Rewards	12
8.3 Token Launch Gates	13
9. Security	13
9.1 Halborn Audit	13
9.2 Threat Model	13
9.3 Quantum Computing Timeline	14
10. Roadmap	14
11. Conclusion	15

ABSTRACT

The advent of cryptographically relevant quantum computers poses an existential threat to every classical cryptographic wallet in existence. Billions of dollars in digital assets secured by ECDSA and RSA will become retroactively exposed the moment Shor's algorithm runs at scale — a scenario cryptographers call "harvest-now-decrypt-later." Veil Protocol is the first production neobank designed from day zero on NIST-standardized post-quantum cryptography: ML-KEM-768 (FIPS 203) for key encapsulation and ML-DSA-65 (FIPS 204) for digital signatures. Combined with biometric self-sovereign identity, a private AI agent powered by fully-homomorphic-encrypted Octra Circles, and an HTTP-native quantum-resistant payment standard (x402-pqc), Veil delivers a complete financial stack that survives the quantum transition — without seed phrases, custodians, or classical key exposure.

1
Introduction

1.1 The Harvest-Now-Decrypt-Later Threat

Nation-state adversaries are intercepting and archiving encrypted traffic today with the explicit intention of decrypting it once a cryptographically relevant quantum computer exists. This "harvest-now-decrypt-later" (HNDL) attack requires no immediate cryptographic capability — it is purely a storage problem that governments and well-funded actors have already solved. Intelligence agencies have signaled that HNDL collection has been operational for years.

For financial assets, the consequences are severe. A private key for an ECDSA wallet observed today can be derived from any historical transaction signature once Shor's algorithm runs at scale. Entire wallet histories — every address, every balance, every counterparty — become transparent. The attack is retroactive: funds held in classical wallets today are already compromised in the adversary's timeline.

1.2 NIST PQC Standards (2024)

In August 2024, NIST finalized the first post-quantum cryptographic standards:

Standard	Algorithm	Purpose	Security Level
FIPS 203	ML-KEM-768	Key Encapsulation Mechanism	Level 3 (AES-192 equiv.)
FIPS 204	ML-DSA-65	Digital Signature	Level 3
FIPS 205	SLH-DSA	Hash-based Signature	Level 3
			ML-KEM (Module Lattice Key Encapsulation Mechanism) is

based on the hardness of the Module Learning With Errors (MLWE) problem — believed to be intractable for both classical and quantum computers. ML-DSA provides digital signatures with the same lattice-based security guarantees. Veil ships on both from day one.

1.3 Why Crypto Wallets Are Vulnerable

Current hardware and software wallets protect private keys from classical adversaries but are structurally unable to address the quantum threat:

Classical wallet vulnerability surface: (1) ECDSA private keys derivable from any on-chain signature via Shor's algorithm; (2) BIP-39 seed phrases stored in plaintext or classical encryption; (3) HD wallet derivation (BIP-32/44) uses HMAC-SHA512 — not quantum-resistant for key material; (4) TLS transport layers rely on ECDH/RSA handshakes harvestable today.

Veil replaces every layer of this stack. Keys are never derived from a mnemonic phrase. Transport uses hybrid KEM-DEM with ML-KEM-768. Signatures use ML-DSA-65. Identity is

derived from biometric data locally, never transmitted.

2

Architecture Overview

Veil Protocol is organized in three interdependent layers, each independently quantum-resistant and composable with the others.

LAYER 3 — INTELLIGENCE / Ghost AI Agent

DistilBERT Intent Classifier · De-identification Pipeline · FHE Circle Execution (Octrä) · Tiered Pricing Engine

LAYER 2 — TRANSPORT / PQCTransport

ML-KEM-768 (FIPS 203) · ML-DSA-65 (FIPS 204) · AES-256-GCM · Replay Window Protection

LAYER 1 — SETTLEMENT / Base + Octrä

x402PQCPayments (Base Mainnet) · VEIL Token / Treasury · Octrä FHE Circles · Gnosis Safe 2-of-2

BIOMETRIC SSI — LOCAL KEY DERIVATION (FUZZY EXTRACTOR + HKDF-SHA3-512)

Layer 1 — Settlement handles asset finality and payment recording on-chain. The x402PQCPayments contract on Base mainnet provides a non-custodial payment ledger. Octrä's FHE Circles provide private computation on the settlement layer without revealing transaction details to any node operator.

Layer 2 — Transport is PQCTransport, a session-oriented cryptographic channel using hybrid KEM-DEM construction. Every byte leaving a Veil client is encrypted under a freshly negotiated ML-KEM-768 session key and authenticated with ML-DSA-65 signatures.

Layer 3 — Intelligence is the Ghost AI Agent layer, which runs DistilBERT intent classification locally, applies de-identification before any LLM call, and executes financial operations inside Octrä FHE Circles.

3

Biometric SSI

3.1 The Seed Phrase Problem

BIP-39 mnemonic phrases are a usability disaster masquerading as a security primitive. They require users to securely store 12–24 human-readable words that, if obtained by any adversary, grant permanent irrevocable access to all funds. Biometric SSI eliminates the seed phrase entirely.

3.2 Fuzzy Extractor

A fuzzy extractor is a cryptographic primitive that extracts a stable, uniformly random key from noisy biometric data. Given a face scan bio_1 and the same face scanned under different lighting or angle as bio_2 , a fuzzy extractor outputs the same key K as long as they are within a defined Hamming distance threshold. The construction uses a secure sketch to publish helper data P — a value that reveals nothing about K but allows reconstruction by the legitimate biometric owner.

```
KeyDerivation(bio: BiometricTemplate):
    sketch = SecureSketch.gen(bio)           // helper data, public
    seed   = ExtractSeed(bio, sketch)        // stable random seed
    ikm    = HKDF-SHA3-512(seed, "veil-v1")
    sigKey = ML-DSA-65.keygen(ikm)
    kemKey = ML-KEM-768.keygen(ikm)
    return { sigKey, kemKey, sketch }
```

3.3 HKDF-SHA3-512 Key Derivation

The extracted seed is passed through HKDF (RFC 5869) using SHA3-512 as the hash function. SHA3-512 is quantum-resistant in that Grover's algorithm reduces its effective security from 512 bits to 256 bits — still well above any foreseeable threat. Separate HKDF contexts produce independent key material for ML-DSA-65 signing keys and ML-KEM-768 encapsulation keys.

3.4 Social Recovery (SSS)

For users who opt into guardian-based recovery, Veil implements Shamir Secret Sharing (SSS). The seed is split into n shares distributed to trusted contacts; any k of n shares allows reconstruction. Shares are individually encrypted to each guardian's ML-KEM-768 public key so no single guardian has access to the underlying seed.

Privacy guarantee: No biometric data is ever transmitted to Veil servers. The fuzzy extractor, HKDF derivation, and key generation all execute locally on-device inside a secure enclave. Veil has zero knowledge of user biometrics.

PQCTransport

4.1 Design Goals

PQCTransport is Veil's quantum-resistant session layer, published as @veil_/pqc-wallet. It replaces TLS's ECDH-based handshake with ML-KEM-768 while preserving the familiar session-oriented API. Design goals: (1) forward secrecy through ephemeral KEM keypairs, (2) authentication through ML-DSA-65 signatures, (3) confidentiality through AES-256-GCM, and (4) replay attack protection.

4.2 ML-KEM-768 (FIPS 203)

ML-KEM-768 provides IND-CCA2-secure key encapsulation at NIST security level 3.

Parameter	Value
Public key size	1,184 bytes
Secret key size	2,400 bytes
Ciphertext size	1,088 bytes
Shared secret size	32 bytes
Security level	NIST Level 3 ("HAES-192)

4.3 ML-DSA-65 (FIPS 204)

Every PQCTransport message is signed with the sender's ML-DSA-65 identity key, derived from the biometric SSI layer. Recipients verify the signature before processing payload content. This binding between transport authentication and biometric identity means that message forgery requires compromising both the ML-KEM session and the signer's biometric — an extremely high bar.

4.4 Hybrid KEM-DEM Construction

```
PQCTransport.send(payload, recipientKemPubKey, senderSigKey):
    // KEM phase
    (ct, ss) = ML-KEM-768.encapsulate(recipientKemPubKey)
    sessionKey = HKDF-SHA3-512(ss, nonce || "pqct-v1")

    // DEM phase
    nonce      = randomBytes(12)
    ciphertext = AES-256-GCM.encrypt(sessionKey, nonce, payload)

    // Authentication
    sig = ML-DSA-65.sign(senderSigKey, ct || nonce || ciphertext)

    return { ct, nonce, ciphertext, sig }
```

4.5 AES-256-GCM

Payload encryption uses AES-256-GCM with a randomly generated 96-bit nonce. GCM provides both confidentiality and integrity — the authentication tag is verified before any plaintext is returned. AES-256 provides 128 bits of security against Grover's algorithm on quantum computers, well above any foreseeable threat.

4.6 Replay Window Protection

PQCTransport maintains a 64-bit sliding-window nonce tracker per session. Incoming messages with a nonce outside the window or already seen within the window are silently dropped. This prevents replay attacks even if an adversary captures and re-injects valid ciphertexts at the network layer.

Ghost AI Agent

5.1 Overview

Ghost is Veil's built-in AI financial agent. It accepts natural-language instructions, classifies user intent, de-identifies the request before any LLM call, executes financial operations inside Oetra FHE Circles, and returns results — all without exposing user financial data to any LLM provider.

5.2 DistilBERT Intent Classifier (96.7% accuracy)

Ghost's first layer is a locally-running DistilBERT model fine-tuned on financial intent classification. The classifier categorizes user input into intents such as `send_payment`, `check_balance`, `swap_token`, `analyze_portfolio`, and `general_query`. Running locally means intent classification never touches an external API. The model achieves 96.7% accuracy on the labeled test set.

Intent Class	Example Input	Tier
check_balance	"What's my ETH balance?"	Free
send_payment	"Send 50 USDC to Alice"	Standard
swap_token	"Swap 0.1 ETH for USDC"	Standard
analyze_portfolio	"How is my portfolio performing?"	Premium
general_query	"Explain impermanent loss"	Free

5.3 De-identification Pipeline

Before any request reaches an external LLM, Ghost applies a multi-stage de-identification pipeline:

Stage 1 — Entity Extraction: NER identifies wallet addresses, amounts, token symbols, and counterparties. These are replaced with typed placeholders: 0x1234...abcd !' [WALLET_A], 500 USDC !' [AMOUNT_1].

Stage 2 — Context Stripping: Historical transaction data, balance figures, and account identifiers are removed from LLM context. The LLM sees only structural intent, not financial specifics.

Stage 3 — Re-identification on Return: LLM responses are post-processed to re-inject original entities before display. The LLM never processes real financial data.

5.4 Tiered Pricing

Tier	Price / Request	Capabilities
Basic	\$0.002	Balance checks, transaction history, simple queries
Standard	\$0.010	Payments, swaps, token transfers, portfolio view
Premium	\$0.050	Portfolio analysis, DeFi strategy, multi-step operations

5.5 FHE Circle Execution (Octra)

Financial operations are executed inside Octra Circles — FHE-encrypted execution environments where computation occurs on encrypted data. Octra Circles allow Ghost to read balances, compute swap routes, and initiate transactions without any Octra node operator being able to observe the underlying data. The RPC endpoint is <https://octra.network/rpc> via JSON-RPC 2.0.

x402-pqc Payment Standard

6.1 HTTP 402 as a Payment Primitive

HTTP status code 402 ("Payment Required") was reserved in the original HTTP/1.1 specification for future use as a machine-native payment signal. The x402 Foundation has proposed a revival of this mechanism as a lightweight payment layer for AI agents and autonomous services. Veil's x402-pqc extension adds quantum-resistant cryptography to the x402 flow.

6.2 Protocol Flow

```
// Standard x402-pqc flow
1. Client !' Server: GET /api/resource
2. Server !' Client: HTTP 402 Payment Required
   Headers:
     X-Payment-Required: amount=0.01, asset=USDC, chain=base
     X-KEM-PublicKey: <ML-KEM-768 public key, base64>
     X-Payment-Version: x402-pqc/1.0

3. Client: negotiate session key via ML-KEM-768
   (ct, ss) = ML-KEM-768.encapsulate(serverKemPub)
   paymentNonce = HKDF(ss, "x402-payment-v1")

4. Client !' Server: GET /api/resource
   Headers:
     X-Payment-Proof: <signed payment record, ML-DSA-65>
     X-KEM-Ciphertext: <ct, base64>
     X-Payment-Nonce: <paymentNonce, hex>

5. Server: verify signature, verify payment on-chain
   !' HTTP 200 + resource
```

6.3 ML-KEM-768 Payment Headers

The payment negotiation uses ML-KEM-768 to derive a shared payment nonce that cryptographically binds the payment proof to the session. This prevents replay: a captured X-Payment-Proof header from one session cannot be replayed in another because the KEM ciphertext and derived nonce are session-specific and one-time.

6.4 AI Agent Discovery via A2A

Ghost agents discover x402-pqc-enabled services via the Agent-to-Agent (A2A) protocol. Services publish an Agent Card at `/.well-known/agent.json` advertising their KEM public key, accepted payment assets, pricing tiers, and capability manifest. Ghost reads Agent Cards autonomously and can initiate paid API sessions without human intervention, using the user's pre-authorized spending limits.

6.5 x402 Foundation Proposal #2664

Veil has submitted a formal proposal to the x402 Foundation (issue #2664) to extend the

x402 standard with post-quantum payment headers. The proposal defines X-KEM-PublicKey, X-KEM-Ciphertext, X-Payment-Nonce, and X-Payment-Version: x402-pqc/1.0 as an optional PQC extension layer compatible with the base x402 spec.

x402PQCPayments Contract

7.1 Contract Overview

x402PQCPayments is a Solidity smart contract deployed on Base mainnet that provides an immutable, non-custodial payment ledger for the x402-pqc protocol. It records payment proofs on-chain, enabling any party to verify that a payment occurred without exposing amounts or counterparties beyond what the blockchain makes visible.

Field	Value
Network	Base Mainnet
Address	0x8F446afA9877C79F3CCb5eaA5b6503752817223f
Owner	0xdEaD1f7583DEFE7A7fD701ea04ba49C14f871a0b (Gnosis Safe)
Deployed	2026-06-21 · Block 47,635,981
Gas used	411,897

7.2 Non-Custodial Design

The contract never holds user funds. It is a pure payment ledger — `registerPayment()` records that a payment occurred; it does not escrow, route, or manage assets. Settlement occurs at the token/USDC layer; the contract provides the cryptographic record. A contract exploit cannot drain user funds because there are no funds to drain.

7.3 Gnosis Safe 2-of-2 Multisig & `renounceOwnership()` Override

Contract ownership is held by a Gnosis Safe with a 2-of-2 signing threshold. No single key can execute owner-only functions. The standard OpenZeppelin `renounceOwnership()` function is overridden to always revert, preventing permanent owner lockout.

The x402PQCPayments contract is the ONLY Veil contract on Base mainnet. All other contracts (VEILToken, VEILTreasury, VEILVesting, VEILNodeRegistry, VEILPaymaster) remain on Base Sepolia testnet pending the Code4rena security audit required for token launch.

Token Economics

8.1 VEIL Token — Fixed Supply

VEIL is a fixed-supply utility and governance token with a hard cap of 1,000,000,000 (1 billion) tokens. There is no inflation mechanism. No additional tokens can ever be minted. The supply is set at contract deployment and the mint function is permanently disabled thereafter.

8.2 Allocation

Category	Allocation	Tokens	Vesting
Ecosystem / Rewards	35%	350,000,000	Ongoing, revenue-backed
Team & Advisors	20%	200,000,000	1yr cliff, 3yr linear
Public Sale	15%	150,000,000	TGE unlock
Treasury	15%	150,000,000	DAO-controlled
Private / Seed	10%	100,000,000	6mo cliff, 2yr linear
Liquidity	5%	50,000,000	TGE unlock

8.3 Revenue-Backed Rewards

Ecosystem rewards are funded by protocol revenue — Ghost AI agent fees, x402-pqc payment processing fees, and node operator staking fees — rather than inflationary token issuance. Reward sustainability is tied to actual protocol usage, not a mint schedule. As protocol revenue grows, the reward pool grows proportionally without diluting existing holders.

8.4 Token Launch Gates

Gate 1 — Revenue: Protocol must demonstrate \$50,000 MRR across Ghost AI fees and x402-pqc payment volume, verified across at least two consecutive months.

Gate 2 — Audit: Code4rena competitive audit of all mainnet-bound smart contracts must complete with all critical and high findings resolved.

9.1 Halborn Audit

Item	Status
MNDA	Signed
Audit status	In Progress
Auditor	Halborn
Scope	Smart contracts + PQC cryptographic layer
Public report	Pending completion

9.2 Threat Model

Classical adversary: Addressed by standard cryptographic hygiene — no plaintext key storage, authenticated encryption, secure random generation, and tamper-evident audit logs.

Harvest-now-decrypt-later adversary: Addressed by ML-KEM-768 (FIPS 203) for all key exchanges and ML-DSA-65 (FIPS 204) for all signatures. Historical ciphertexts cannot be decrypted by a future quantum computer.

Biometric adversary: Addressed by fuzzy extractor design — the secure sketch reveals no information about the underlying key without the original biometric, and biometric data never leaves the device.

9.3 Quantum Computing Timeline

Expert consensus from NIST, NSA (CNSA 2.0 Suite), and academic researchers places the emergence of cryptographically relevant quantum computers (CRQCs) capable of breaking 256-bit ECC between 2030 and 2040, with increasing probability of earlier arrival as quantum error correction improves. The HNDL threat is already active — data collected today is the adversary's backlog for tomorrow.

Veil's PQC implementation is algorithm-agile: if ML-KEM-768 or ML-DSA-65 are later found to have unexpected weaknesses, the transport and identity layers can be re-keyed to alternative NIST-approved algorithms (SLH-DSA, FN-DSA) without requiring users to re-create accounts or re-enroll biometrics.

Roadmap

Phase	Timeline	Key Milestones
SDK & Agents	Live — Q2 2026	@veil_/* packages on npm. x402PQCPayments on mainnet. Ghost AI MCP server.
Mobile	Q3 2026	iOS/Android Expo app. On-device DistilBERT. Full x402-pqc flows.
Enterprise & Token	Q4 2026	Token launch (gated). Enterprise SDK. VEILNodeRegistry mainnet. B2B API.

Conclusion

The quantum transition is not a distant hypothetical — it is an active adversarial campaign with a multi-decade payoff horizon. Every classical wallet deployed today is a ticking vulnerability for the harvest-now-decrypt-later threat.

Veil Protocol demonstrates that quantum-resistant cryptography, biometric self-sovereign identity, and private AI execution are not mutually exclusive with user experience. By combining NIST-standardized ML-KEM-768 and ML-DSA-65 at every layer — transport, identity, and payment — Veil delivers a financial stack that is secure against both today's classical adversaries and tomorrow's quantum adversaries.

The open-source packages are available today for any developer building on the post-quantum stack. The x402-pqc standard is an open proposal. The future of money is lattice-based.

RESOURCES

GitHub: github.com/veil-protocol-1/veil-pqc
 npm: @veil_/pqc-wallet · @veil_/auth · @veil_/x402-pqc · @veil_/circles · @veil_/agent-registry
 x402: github.com/x402-foundation/x402/issues/2664
 Web: veilprotocol.net

